CSE 474 Final Project Report

Pokemon Identification Software With Image Classification

Option 2: ML Model in a New Use Case Setting

By Eric Ohebshalom

Abstract:
The goal of this project is to create a machine learning image recognition model that is trained to identify images of Pokemon species. I created a preliminary model with AlexNet with only a small handful of pokemon, as well as a large model with ResNet with almost all of the pokemon. Overall, the model is successful at doing its job but struggles with newer pokemon and pokemon that look very similar to other pokemon.

Introduction:
As we all know, Pokemon is a popular and ever growing multi-media franchise that has existed for 30 years with no signs of slowing down. There are video games, anime, movies, trading cards, a lot of merchandise, and so much more. It is currently the highest grossing media franchise of all time and in most years, the empire makes more money than the year prior displaying its evergreen growth. The highlight of the Pokemon franchise are a variety of monsters known as Pokemon. While there are Pokemon species like Pikachu and Charizard that everyone knows about, there are also many lesser known ones that a newer or less invested fan may not be familiar with. As of 2026, 1028 pokemon species have been created and even more when you include alternative forms. This can be overwhelming and intimidating for anyone getting into the franchise for the first time so it would be very useful if there was an application that takes an image of a pokemon as an input and outputs the user with the Pokemon species's name and some information about it. Funnily enough, one of the very first things a player receives when they first start a pokemon game is an in game device known as the Pokedex. This device does exactly what I'm trying to create, identifying pokemon through images and when the franchise started in the 90s, it was seen as a high tech fantasy device. Since then, image recognition machine learning models have greatly progressed and with so many variations of all 1028 pokemon available on the internet to train on, the concept of this device can now become a reality.

Dataset:
With such a large variety of different media the pokemon franchise has tackled, all 1028 pokemon have many images available on the internet that can be used to train on. The dataset I will be using is the *Ultimate Pokémon Images & Updated Pokédex Dataset* from Kaggle which is a dataset that contains a total of 130,000 images of 1025 pokemon from all generations aside from the 3 generation 10 pokemon that have been revealed. The dataset contains 1069 classes as mega evolutions are considered separate in the dataset. Each pokemon species has images with a variety of poses, art styles, and backgrounds which is necessary for an image classifier that needs to be able to recognize what a pokemon looks like from a pose or art style that the model has never seen before. It also contains both images that are labeled with the pokemon as well as unlabeled images.

The dataset will be divided between a training, testing, and validation set where the training set will get 80% of the images while the other 2 each get 10% of the images. Potential problems in the dataset include pokemon that look very similar. Some pokemon of different species look very similar because they are a part of the same evolutionary line(baby/adult forms of other pokemon). For example, the only difference between the pokemon Finizen and the base form of Palafin is that Palafin's stomach adds a heart. The model would have to distinguish between the 2 and this is an example of inter-class variability. Likewise, there are many pokemon with alternative forms that look very different from their base form. The aforementioned Palafin has a hero form that changes its color and most of its design. The model would have to classify both forms under the same class while also not confusing the base form with Finizen. This is an example of intra-class variability

Finizen Palafin(Base) Palafin(Hero)

Finally, not all pokemon are treated equally. For example, an old and iconic pokemon like Pikachu has 570 labeled images in the dataset while a new and less iconic pokemon like rellor only has 10. This disparity in data may result in the model being less accurate for the pokemon that more people will have trouble recognizing.

Existing Works:

Image classification machine learning models are nothing new. The core of this project is Fine-Grained Visual Classification(FGVC) which is a process that distinguishes different variations of objects that belong to a common big class. In this case, we want the model to distinguish between different species of pokemon rather than just declaring everything to be a Pokemon or not. As discussed earlier with Finizen and Palafin, small inter-class and large intra-class variations are challenges that FGVC tries to overcome. Early FGVC models mostly used Convolutional Neural Networks(CNN) but as time went on, visual transformer architectures became more commonplace. All the advancements have even caused many people to create their own pokemon identification models like Pokepedia and Pokémondium.

References:

Foundation of FGVC:
Fu, J., Zheng, H., & Mei, T. (2017). "Look Closer to See Better: Recurrent Attention Convolutional Neural Network for Fine-Grained Image Recognition." *CVPR*.
He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep Residual Learning for Image Recognition." *CVPR*.
Huang, S., Xu, Z., Tao, D., & Zhang, Y. (2016). "Part-Stacked CNN for Fine-Grained Visual Categorization." *CVPR*.
Lin, T. Y., RoyChowdhury, A., & Maji, S. (2015). "Bilinear CNNs for Fine-grained Visual Recognition." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
Zheng, H., Fu, J., Mei, T., & Luo, J. (2017). "Learning Multi-Attention Convolutional Neural Network for Fine-Grained Image Recognition." *International Conference on Computer Vision (ICCV)*.

Modern Transformers & Attention Mechanisms:

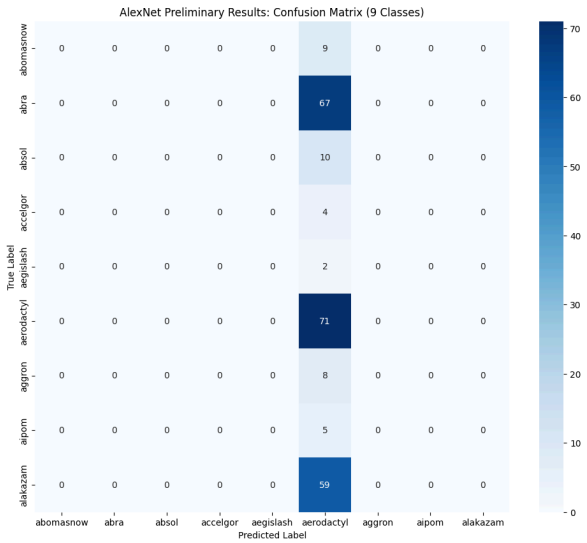
Dosovitskiy, A., et al. (2020). "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale." *ICLR*.
He, S., et al. (2022). "TransFG: A Transformer-based Framework for Fine-grained Visual Classification." *CVPR*.
Luo, W., et al. (2024). "Dual-Dependency Attention Transformer for Fine-Grained Visual Classification." *PMC - NIH*.
Li, S., et al. (2025). "Fine-Grained Visual Classification via Adaptive Attention Quantization Transformer." *PubMed*.
Radford, A., et al. (2021). "Learning Transferable Visual Models From Natural Language Supervision."

Pokemon Image Classification Approaches
Sampaio Campos, F. (2024). "Building a Pokémon Image Classifier using Deep Learning in PyTorch." *Medium/Technical Report*.
Chirra, V. R., et al. (2022). "Pokepedia: Transfer Learning for Pokémon Image Classification." *Journal of Networking and Network Applications*
Chu, Y., et al. (2023). "Attribute Prediction in Pokémon Images using CNNs." 2nd International Conference on Electrical Engineering and Big Data Analytics (EEBDA).
Zhang, L. (2025). "Pokémon Classification Using Convolutional Neural Networks: A Deep Learning Approach for Fine-Grained Image Recognition."
Soni, R., (2026). "Pokémondium: A Machine Learning Approach to Detecting Images of Pokémon." ResearchGate.

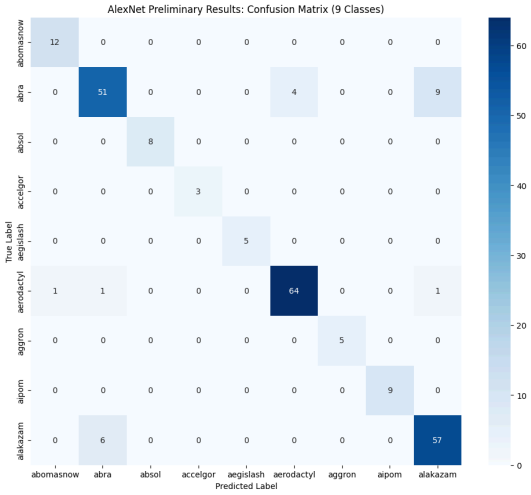
Preliminary Approach:
To address the challenges of classifying images into over a thousand categories, I will use and compare two different CNN architectures. One more foundational, and one more modern. AlexNet is the foundational architecture. AlexNet was created in 2012 and is credited for popularizing deep CNNs. This model may struggle a little more with the inter and intra class variabilities as it only has 8 layers of depth and will have a harder time identifying finer details. On the other hand, ResNet is a more advanced architecture created in 2015 and its shortcut connections allows it to have a higher depth and more layers allowing for a greater attention to fine details and thus struggling less with the inter and intra class variabilities that AlexNet struggles with. The models will be implemented with Pytorch using the Adam optimiser with a Cross-Entropy Loss function which is the standard for multi class classification tasks like image classification. In the models, the dataset will be split 80% training, 10% validation, and 10% testing. The model will adjust its parameters using the training data, check its correction between epoch loops with the validation data, and do 1 final check with the testing data to ensure that the final model works with data it has never seen. After checking with the testing data, the model will generate a confusion matrix to see which pokemon the model succeeds and struggles with as well as which pokemon the models mix up the most

Preliminary Results:
For now, I will build a small AlexNet model on Google Colab that is only trained on images of 9 pokemon. The pokemon are Abomasnow, Abra, Absol, Accelgor, Aegislash, Aerodactyl, Aggron, Aipom, and Alakazam. The pokemon have a variety of designs, have a wide range of iconicity, includes pokemon with different forms, and even includes Abra and Alakazam which are 2 pokemon that are a part of the same evolutionary line. After 10 rounds of training

and validations, the model finished with a score on the test set of ~35%. While not great, this is an improvement over 11% which is what would happen if the model was just guessing 1 out of the 9 classes randomly. Then I generated the confusion matrix and realized that the model was only guessing Aerodactyl, who has the most training data out of all the pokemon in the sample. The model learned that if it just guessed Aerodactyl, it would minimize the most losses so that's what it did. Aerodactyl has 696 images in the data while something like Accelgor only has 35 as Aerodactyl is older, and is overall the most iconic



I then lowered the training rate from 0.001 to 0.0001 and adjusted the weights so that pokemon with less training data are punished more when the model gets them wrong. This corrected everything and now the model had a much better accuracy of ~90% with a much better confusion matrix where most guesses are along the diagonal.



The only minor confusion is between Abra and Alakazam and this is expected as they are a part of the same evolutionary line and thus have similar designs. Overall, this shows an amazing proof of concept and a good jumping off point for the rest of the project.

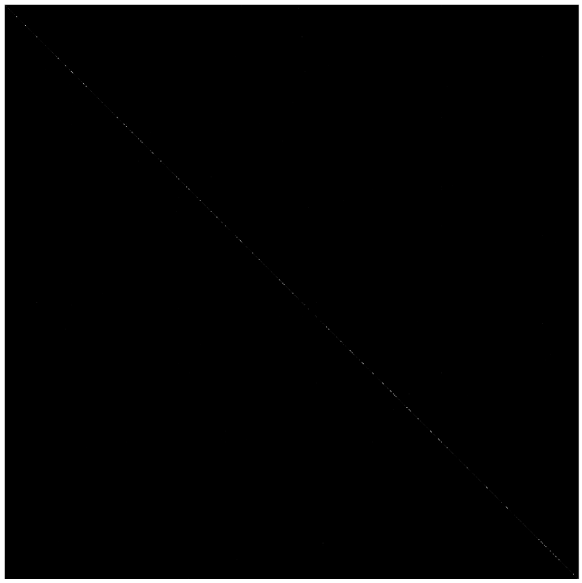
Final Approach:

Going from only 9 pokemon species to over 1000 is a big shift and resulted in a lot of unexpected hurdles. The first major hurdle that I didn't expect to be as hard as it was was getting all the data into google colab in the first place. With only 9 classes, getting the data into Google Colab was as simple as dragging and dropping from my desktop into the browser and this was feasible as the size of the dataset was much smaller. Now, the total size of all the data was over 10GBs so that's no longer feasible as it would take too long. To get all the data into Colab, I had to compress all the data into a zip file, put the zip file into my Google Drive, write code that runs commands that transfers the zip file from Drive into Colab, and finally uncompress all of the data into Colab's content directory but with that done, I can now work on the final model. As explained earlier, the model's architecture

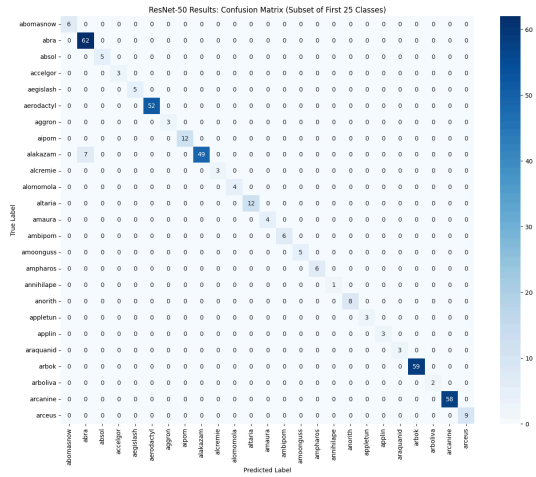
has switched from AlexNet to ResNet which is better at recognizing the more minute details and is much better when it comes to larger and deeper neural network models and I also upgraded the hardware used for training. The preliminary model was light enough to be trained on just the CPU that comes with the free tier of Google Colab with batch sizes of only 32 and while training wasn't the fastest, it was good enough to get the job done. With a model that's 100 times the scale, this approach wasn't gonna cut it as training would take hours upon hours. An A100 GPU was needed but this was powerful enough to allow batch sizes of 256 for the training set and 512 for the validation and testing sets. Previous batch sizes were only 32 so this dramatically sped up training and was necessary for scaling up the project. The basic steps for creating the model are mostly similar to the preliminary model only with complications due to the very large dataset. The data was still split between 3 subsets of 80% training, 10% validation, and 10% testing but with so many classes and a wide range of the number of images within a class combined with the random nature of how the data is split, there would inevitably be instances where a specific class wasn't given a datapoint in one of the 3 subsets. On my first few attempts at training the model, a few pokemon species were never trained properly as they didn't have any images in the validation set or they weren't recorded in the final results as they didn't have any images in the testing set. To fix this, the new code had to first manually pick 3 images in each class to put in each of the 3 subsets and then do the 80/10/10 split on the rest of the data to guarantee that each class had at least 1 datapoint in each subset. But with all of that out of the way, I can now train the final model.

Final Data:

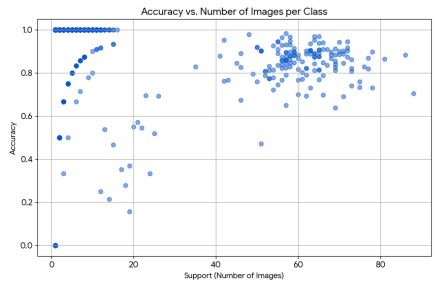
After 10 epochs of training, the accuracy for the training set kept increasing but the accuracy for the validation set stayed at ~87% indicating that overfitting was starting to occur and the model was starting to focus too heavily on the training set. At this point, I ended training and tested the model with one final check on the testing set which had an accuracy of 88.38%. While not perfect, this does demonstrate that the model can identify images of pokemon a majority of the time. Just like with the preliminary results, the code generated a confusion matrix to see the ratios between what the model guessed and what the species actually is but with over 1000 classes, it's not the easiest thing in the world to look at and gain data from.



For easier visualization, I generated a subset of the confusion matrix that only contains the first 25 pokemon alphabetically including the 9 pokemon that were in the preliminary model.



With most of the white being on the diagonal of the matrix, we can see that the model predicts the correct species most of the time. While the model still sometimes guesses abra when it was really alakazam, it never did the opposite showing signs of progress in terms of inter class variability. The confusion matrix also demonstrates just how big of a gap there is between the amount of data points and how that influences results. Abra, Aerodactyl, Alakazam, Arbok, and Arcanine are all Generation 1 pokemon and thus had by far the most images in the dataset.
The code also generated a CSV file with all of the data and there we can see all class precision(the accuracy for if model was correct when it guessed a specific class), recall(how accurate the model was when show an image of a specific pokemon), F1 score(a combination of recall and precision), and support(the amount of times the class was in the testing set).



Newer pokemon with less datapoints often had a 100% f1 score but this was misleading as it was mainly because the model had to identify a lot fewer of them in the testing set and was much more likely to get all of them correct when compared to an older pokemon with more datapoints. 16 classes only had 1 image in the testing set and they got it wrong meaning that those had an accuracy of 0% and all of them were either pokemon from the newest generation or mega evolutions(we'll get to them later).

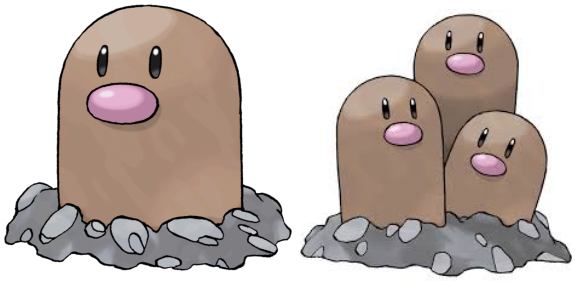
	precision	recall	f1-score
miraidon	0	0	0
mega_steelix	0	0	0
mega_scizor	0	0	0
mega_rayqua	0	0	0
mega_metag	0	0	0
mega_medic	0	0	0
mega_lopunr	0	0	0
mega_latias	0	0	0
mega_hound	0	0	0
mega_heracr	0	0	0
mega_garcho	0	0	0
mega_audinc	0	0	0
mega_aggron	0	0	0
iron_jugulis	0	0	0
finizen	0	0	0
dipplin	0	0	0

As previously expected, a lot of the confusions came from pokemon in the same evolutionary line that had similar designs and it didn't help that many older pokemon have very simple designs due to hardware limitations of the original GameBoy. Remember that the model had an overall accuracy of 88.38% with the testing set which can be used as a baseline for how good or bad an F1 score is.

Omanyte and Omastar both had an F1 score of ~77.5%.

Clefairy and Clefable were both ~80%

Diglett had ~83% Dugtrio had ~80%



The biggest confusions came from confusing mega evolutions with their base form. Since the original dataset differentiated mega evolutions with their base form, the model put them into separate classes but mega evolutions often have very similar designs and poses with the base form, even more than a regular evolution leading to a lot of confusion. The base form is older and thus has more data in the dataset so if the model sees an image of the mega evolution, it often just predicts the base form as they've seen a very similar design a lot more often.

Mega Beedrill had F1 score of ~41%

Mega Kanghaskhan had F1 score of ~18%

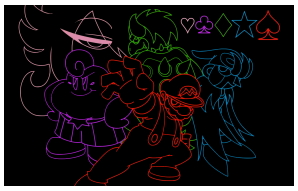
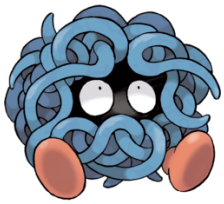
Charizard and Mewtwo both have 2 mega evolutions so this leads to 3 way confusions

Mega Charizard X had ~47%. Mega Charizard Y had ~26%

Mega Mewtwo X had ~35%. Mega Mewtwo Y had ~21%

And as mentioned earlier, 12 mega evolutions had an f1 score of 0%.

For fun, I told the model to identify a random unrelated image such as my computer's homescreen. My face was Paras and my homescreen was Tangela and if I had to guess why the model predicted these, my computer's homescreen has a similar shade of black and blue as Tangela and my skin color kind of looks like the color of the skin of Paras. Both of these pokemon are also Generation 1 pokemon and since generation 1 pokemon were the bulk of the dataset used, guessing what it saw the most in training was most likely to get a correct result if nothing could be gathered from the image.


is a


is a

I also gave it a picture of a real life crab as there is a pokemon named Krabby which is literally just a crab(yes very creative) and sure enough, the model predicted Krabby.

is a

Conclusion:
Overall, while a lot could certainly be improved, the model exceeded my expectations. Using a dataset containing more newer pokemon would lead to better overall accuracy among those pokemon and doing things like putting more modifications of already existing images in the datasets would make training better as it would make the model better at images it's never seen before but I feel like this is a good place to call it. The main issue I have with the current model is that the pokemon with more images in the dataset are more likely to be guessed but the pokemon that have the most images are the older and more iconic pokemon and are less likely for people to have trouble recognizing. Everyone knows what a Pikachu and Charizard is so they wouldn't use a device to identify those but those are the pokemon that the model has the least trouble with making the model a little counterintuitive so hopefully, this could be ironed out in the future.